

Part 10: Numerical Determination of MLE

In some situations it is not possible to derive the MLE using an analytical approach, so numerical optimization methods are required.

The primary limitation is that one does not arrive at a general form for the estimator as a function of any data set. Instead, the result is the estimate of the parameter(s) for the one data set under consideration.

When taking a numerical approach, it is still a good idea to work with the log likelihood, as this function is usually better behaved. (The product in the likelihood function will often lead to a likelihood with either very small or very large values.)

Python includes general-purpose optimization capability; here we will focus on `minimize()`, a part of `scipy`, which minimizes a function specified by the user.

```
In [1]: from scipy.optimize import minimize
```

We'll begin with a simple toy example. Suppose that X is binomial(n, p), and that p is unknown. We've seen that the sample proportion $\hat{p} = X/n$ is unbiased and has standard error of $\sqrt{p(1-p)/n}$. What value of p maximizes this standard error? Of course, we know that $0 \leq p \leq 1$.

Start by building the function which is to be minimized.

```
In [2]: import numpy as np

def negseofphat(p, n=100):
    return (-1)*np.sqrt(p*(1-p)/n)
```

Comments on this:

1. The optimization procedure minimizes the provided function, of course, so `negseofphat()` is defined to return -1 times the standard error.
2. It is crucial that the first input argument to the function to be optimized (in this example, `negseofphat()`) be the variable(s) over which the function is to be optimized. If there is more than one such variable, then the variables must come in as an array. We'll do such an example below. In this example, we are minimizing over p , so `p` goes in as the first argument.
3. There can be any number of additional arguments to the function, but these must start with the second in the list. Their values will be passed on to `negseofphat()` through the optimization function.

Now, we'll actually call `minimize()` to do the optimization. See <https://docs.scipy.org/doc/scipy/reference/tutorial/optimize.html> for more information.

```
In [15]: n=100

res = minimize(negseofphat, 0.1,
               method='Nelder-Mead', options={'disp': True},
               args=(n))
```

Optimization terminated successfully.

Current function value: -0.050000

Iterations: 18

Function evaluations: 36

Comments on this:

1. The first argument is the function to be minimized. Note that there are no parentheses; it's simply the name of the function.
2. The second argument to this function (0.2 in this example) is the starting values for the parameter(s) over which the function will be minimized. This should be on the interior (not the boundary) of the space of possible values for the parameter(s). If there is more than one parameter, then this argument should be an array.
3. The `method` argument tells `minimize` which minimization technique to utilize. Here, I am using `Nelder-Mead`, i.e., the classic optimization approach of Nelder and Mead.

4. The `options` argument can set various properties, including the maximum number of iterations, the tolerance, and, as in the example, the amount of information displayed. The available options will vary depending on the `method` chosen.
5. Any arguments which are to be passed on to the function to be minimized are then listed via the `args` syntax.
6. The input value at which the function is minimized is found via the following:

```
In [16]: res.x
```

```
Out[16]: array([0.5])
```

Numerical Maximization of Likelihood in Gamma Case

Next consider the case where $X_1, X_2, \dots, X_n$ are i.i.d. $\text{Gamma}(\alpha, \beta)$. The following function will return the MLE when a vector of observations are passed in.

```
In [17]: def gammamle(x):

    from scipy.stats import gamma

    def negloglikelihood(pars, x):
        return (-1) * sum(gamma.logpdf(x,
            pars[0], scale=1/pars[1]))

    betahatmom = np.mean(x) / (np.mean(x**2) -
        np.mean(x)**2)
    alphahatmom = betahatmom * np.mean(x)

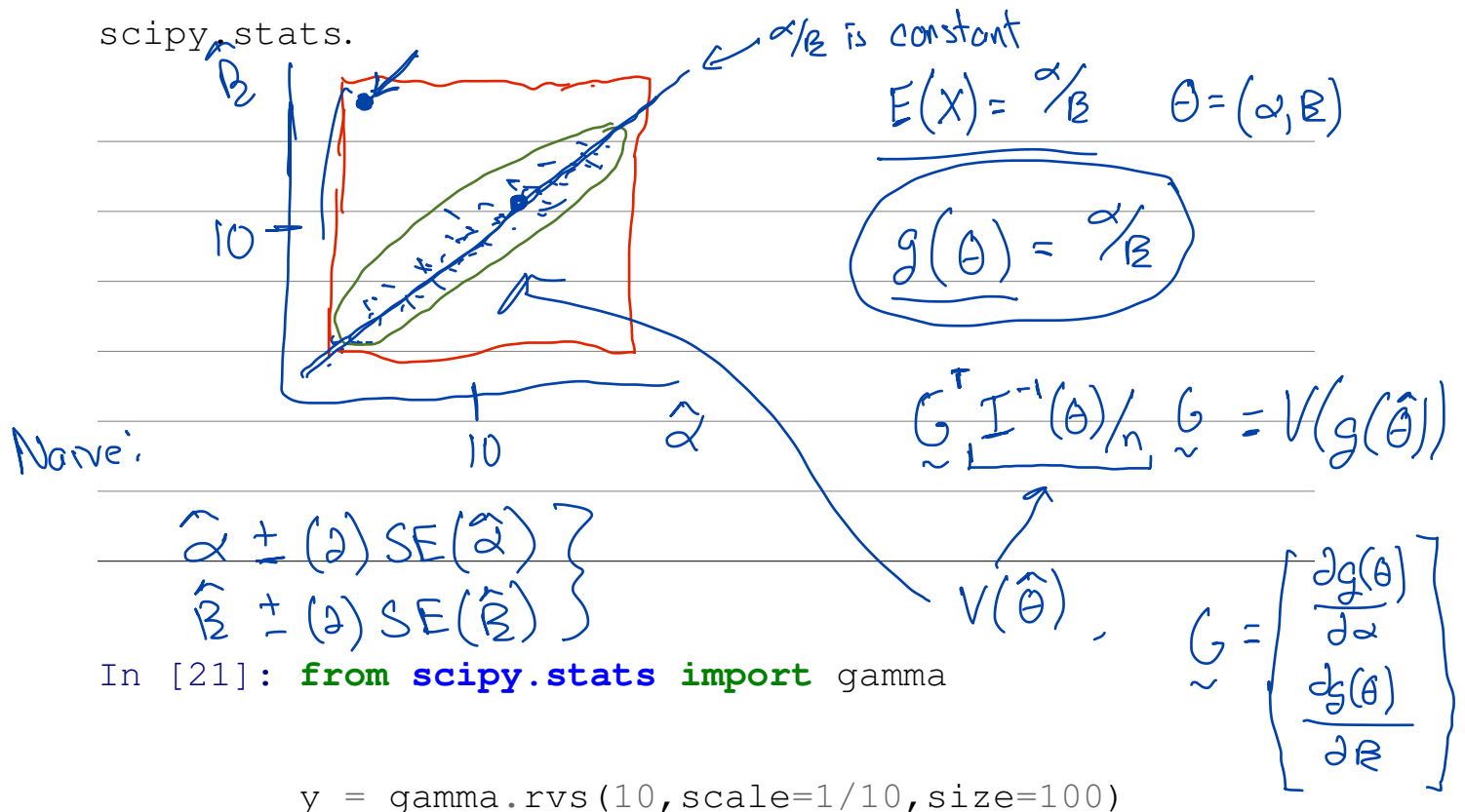
    mleout = minimize(negloglikelihood,
        [alphahatmom, betahatmom], args=(x),
        method="Nelder-Mead")

    return mleout
```

Handwritten annotations in the code block:

- A blue arrow points from the handwritten (α, β) to the `gamma` import statement.
- Below `pars[0]` is a handwritten α .
- Below `scale=1/pars[1]` is a handwritten β .
- A blue bracket groups the calculation of `betahatmom` and `alphahatmom`.
- A blue arrow points from the `args=(x)` in the `minimize` call to the `x` parameter in the `negloglikelihood` function definition.

Exercise: Test `gammamle()` by simulating some data sets where the true values of the parameters are known. Use `gamma.rvs()` from `scipy.stats`.



In [21]: `from scipy.stats import gamma`

`y = gamma.rvs(10, scale=1/10, size=100)`

`gammamle(y).x`

Out[21]: `array([9.54379825, 9.80684911])`

$$g(\theta) = P(a \leq X \leq b)$$

Example: Regression with t-distributed Errors

In this example, we will consider the following regression model: The response Y is related to the single predictor by

$$Y = \beta_0 + \beta_1 x + \sigma \epsilon$$

where ϵ is assumed to have the t -distribution with ν degrees of freedom. We will assume that ν is set by the user (not estimated).

There are three unknown parameters in this model: β_0 , β_1 , and σ . We seek to estimate these via maximum likelihood.

Note that σ is not exactly equal to the standard deviation of the irreducible errors since the t -distribution does not have variance of one. Nevertheless, σ is closely related to the standard deviation of the irreducible errors.

$$T \sim t_\nu, \quad \underline{V(T) = \frac{\nu}{\nu-2}}$$

Exercise: We assume that we will observe n pairs of (x, Y) values, and that the ϵ values are independent. Derive the density for a single Y observation.

Since ϵ_i are independent, Y_i are independent

$$Y_i = \beta_0 + \beta_1 x_i + \sigma \epsilon_i$$

$$F_{Y_i}(y) = P(Y_i \leq y) = P(\beta_0 + \beta_1 x_i + \sigma \epsilon_i \leq y)$$

$$= P\left(\epsilon_i \leq \frac{y - (\beta_0 + \beta_1 x_i)}{\sigma}\right)$$

$$= F_T\left(\frac{y - (\beta_0 + \beta_1 x_i)}{\sigma}\right), T \sim t_v$$

$$f_{Y_i}(y) = \frac{\partial F_{Y_i}(y)}{\partial y} = f_T\left(\frac{y - (\beta_0 + \beta_1 x_i)}{\sigma}\right) \left(\frac{1}{\sigma}\right)$$

$$\text{So, } L(\beta_0, \beta_1, \sigma) = \prod_{i=1}^n f_{Y_i}(y_i)$$

$$= \prod_{i=1}^n f_T\left(\frac{y_i - (\beta_0 + \beta_1 x_i)}{\sigma}\right) \left(\frac{1}{\sigma}\right)$$

residuals

In preparation for using `minimize()`, here is a function that returns the negative log-likelihood as a function of the three parameters.

In [22]: `def neglogliketreg(pars, x, y, nu):`

$\swarrow \beta_0, \beta_1, \sigma$

```

    from scipy.stats import t

    resids = y - (x*pars[1] + pars[0])
    return (-1)*\
        sum(t.logpdf(resids/pars[2], nu)) + \
        len(x)*np.log(pars[2])

```

β_1 β_0

σ

Exercise: Carefully explain the structure of the above function.

The log-likelihood is

$$\log \prod_{i=1}^n f_T\left(\frac{y_i - (\beta_0 + \beta_1 x_i)}{\sigma}\right) \left(\frac{1}{\sigma}\right)$$

$$= \sum_{i=1}^n \log f_T\left(\frac{y_i - (\beta_0 + \beta_1 x_i)}{\sigma}\right) - n \log \sigma$$

This function will fit the model using the assumed t -distributed errors.

In [23]: `def treg(x, y, nu):`

```
    from scipy.stats import t
```

```
    def neglogliketreg(pars, x, y, nu):
        resids = y - (x*pars[1] + pars[0])
        return (-1)*\
            sum(t.logpdf(resids/pars[2], nu)) + \
            len(x)*np.log(pars[2])
```

```
    holdcov = np.cov(x, y)
```

```
    betalinit = holdcov[0,1]/holdcov[0,0]
```

```
    beta0init = np.mean(y) - np.mean(x)*betalinit
```

```
    resids = y - (beta0init + betalinit*x)
```

```
    sigmainit = np.sqrt(sum(resids**2)/len(x))
```

```
    optout = minimize(neglogliketreg,
```

```
        [beta0init, betalinit, sigmainit],
```

```
        args=(x, y, nu), method="Nelder-Mead")
```

```
    return optout
```

MLE
under
normal
assumptions

Exercise: Run simulations to test the above function.

```
In [24]: from scipy.stats import t
```

```
x = np.array(range(100))
y = 4 + 5*x + 10*t.rvs(1, size=len(x))
treg(x, y, 5).x
```

Handwritten notes: B_0 B_1 σ (under 4, 5, and 10 respectively). An arrow points from "Cauchy dist." to the parameter 1 in the `rvs` function.

```
Out[24]: array([ 5.99008462,  4.94991786, 21.41289667])
```

0.0.1 Fisher Information

In cases where the likelihood is maximized numerically, it is unlikely (but not impossible) that the Fisher Information can be derived analytically.

Instead, we can use an approximation based on the result seen previously:

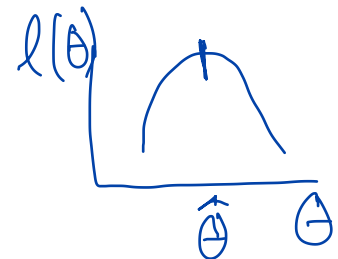
$$nI(\theta) = E \left[- \frac{\partial^2}{\partial \theta^2} \log L(\theta) \right]$$



This result suggests that we can approximate $nI(\theta_0)$ by using the **curvature at the peak of the observed likelihood function**.

This quantity is returned by most numerical optimization routines, called the **Hessian**.

We will call this the **observed Fisher Information**.



Example: To illustrate the idea, let's consider a one-parameter model based on the Gamma distribution. In particular, we assume that $X_1, X_2, \dots, X_n$ are i.i.d. $\text{Gamma}(\alpha, 1)$.

This means that

$$f_X(x; \alpha) = \frac{1}{\Gamma(\alpha)} x^{\alpha-1} e^{-x}, \quad x > 0$$

Recall from our previous discussion that `gamma.logpdf` can be used to evaluate the likelihood evaluated at `alpha`, for example:

```
In [25]: from scipy.stats import gamma
```

```
alpha = 1.3
```

```
x = [1, 2, 3]
```

```
sum(gamma.logpdf(x, alpha, scale=1))
```

```
Out [25]: -5.137947730708002
```

The function `Hessian` as part of `numdifftools` will evaluate the Hessian of a general function. See the example below.

```
In [26]: def onedgammamle(x):
```

data ↙

```

    from scipy.stats import gamma
    import numpy as np
    from scipy.optimize import minimize
    import numdifftools as nd

    def negloglikelihood(pars, x):
        return (-1) * sum(gamma.logpdf(x,
            pars[0], scale=1))
        }

    alphahatmom = np.mean(x)

    mleout = minimize(negloglikelihood,
                      alphahatmom, args=(x),
                      method="Nelder-Mead")

    a function → hessfunc = nd.Hessian(negloglikelihood)
    mlevar = 1/hessfunc(mleout.x, x)
                 $1/n I(\hat{\alpha})$ 
    return [mleout.x, mlevar]
```

Exercise: Comment on the syntax used in this function.

Hessian() returns a function

I want the curvature at peak of log likelihood, so

hessfunc(mleout.x, x)

↑
passed on to
negloglikelihood

Exercise: Consider the code and output below. Interpret what is being found.

```
In [38]: x = gamma.rvs(1, scale=1, size=100)
```

```
onedgammamle(x)
```

```
Out[38]: [array([0.48755953]), array([[0.00194097]])]
```


Exercise: Why did I not need to take the negative of the Hessian in order to find the variance?

The (-1) cancels out with the (-1)
in negloglikelihood
